remembers and sends on subsequent requests to the same site until the cookie expires or is deleted. (You will implement cookie-based storage in the rest of this chapter and learn how to protect cookies against common attacks.) Cookies are still a great choice for first-party clients running on the same origin as the API they are talking to but can be difficult when dealing with 3rd-party clients and clients hosted on other domains. In chapter 5, you will implement an alternative to cookies using HTML 5 local storage that solves these problems, but with new challenges of their own.

4.2.1 A token store abstraction

In this chapter and the next two, you're going to implement several storage options for tokens with different pros and cons, so let's create an interface now that will let you easily swap out one solution for another. Figure 4.5 shows the `TokenStore` interface and its associated `Token` class as a UML class diagram. Each token has an associated username and an expiry time, and a collection of attributes that you can use to associate information with the token, such as how the user was authenticated or other details that you want to use to make access control decisions. Creating a token in the store returns its ID, allowing different store implementations to decide how the token should be named. You can later look up a token by ID, and you can use the `Optional` class to handle the fact that the token might not exist; either because the user passed an invalid ID in the request or because the token has expired.